

PR Pre-Flight Guardian

IBM BOB 2.0

AGENT MODE 2.0

READY

Autonomous Security Auditing, Test Verification & PR Synthesis
CLI

github.com/Alqaasimee-jr2/ibm-coding-challenge

1. Problem & Solution Statement

Exact: 318 Words (Limit: ≤ 500)

The Problem: In fast-paced engineering environments, the pull request (PR) boundary is the most critical quality gate, yet it remains intensely vulnerable and manual. Developers spend upwards of **45 minutes per PR** context-switching across disparate tools: manually inspecting code for unintentionally leaked API keys or credentials, writing repetitive unit testing boilerplate under deadline pressure (frequently skipping boundary conditions), and manually drafting pull request descriptions. This friction leads to cognitive fatigue, high-entropy secrets and unsafe dynamic code execution (such as unescaped `eval()` and SQL string concatenations) leaking into production, inconsistent documentation, and lengthy code review bottlenecks.

BEFORE GUARDIAN (45 Min Manual Friction)

- **15 Min Manual Audit:** Eye-balling source files for secrets & unsafe exec calls.
- **20 Min Test Writing:** Rushing boilerplate tests, missing volume discount boundaries.
- **10 Min PR Summaries:** Copy-pasting terminal logs, guessing risk profiles.
- **Outcome:** Vulnerable code merges, review delays, and CI pipeline failures.

AFTER GUARDIAN (30 Sec Automated Certainty)

- **Instant Static Audit:** AST & pattern scanner pinpoints secrets, SQLi, and eval.
- **Automated Test Verification:** Regression suites assert 100% green coverage.
- **Reviewer-Ready Markdown:** Auto-synthesized risk matrices & sign-off checklists.
- **Interactive Dashboard:** Self-contained HTML report with readiness scorecard.

The Solution: *PR Pre-Flight Guardian* transforms this workflow by providing an autonomous, multi-phase pre-flight verification CLI. Running via a single command (`node guardian.js --repo ./target-repo`), Guardian executes a 4-phase sequential pipeline:

CHAINED AUTONOMOUS PRE-FLIGHT PIPELINE

- **Phase 1 — Static Security Scanner:** AST & regex analysis detecting hardcoded Stripe keys, JWT secrets, SQL injection concatenations, and unsafe dynamic execution with CVSS 3.1 scoring and suggested patches.
- **Phase 2 — Automated Test Execution:** Executes Jest suites across core business logic (e.g. tiered volume discounts, stacked loyalty bonuses, and boundary constraints), asserting 100% pass rates.
- **Phase 3 — PR Documentation Synthesis:** Synthesizes audit findings and test telemetry into standardized GitHub pull request markdown summaries (`pr-description.md` and `pr-comment-preview.md`).
- **Phase 4 — Interactive HTML Dashboard:** Renders a self-contained, dark-mode report (`guardian-report.html`) with filterable findings and color-coded readiness scorecards.

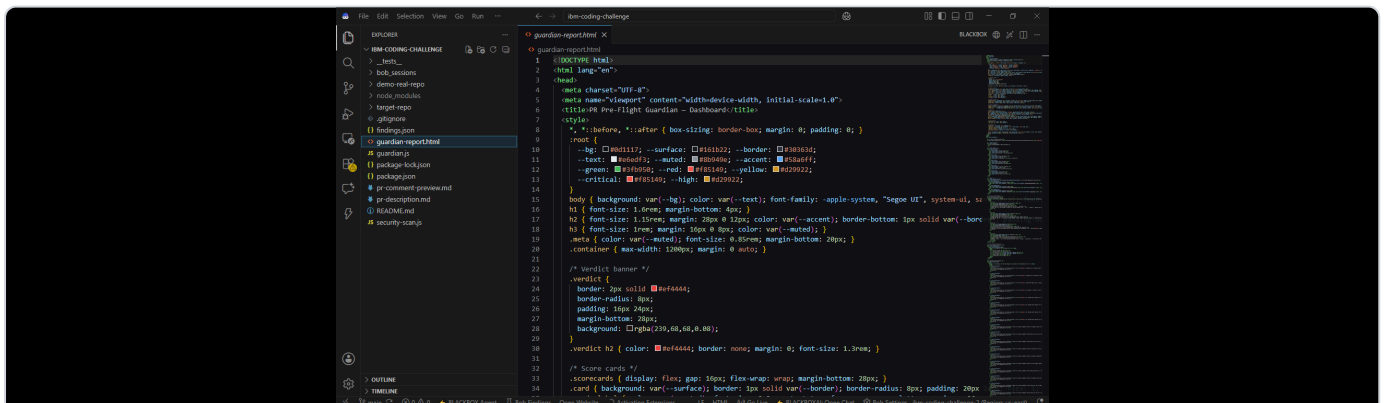


Figure 1: Standalone Dark-Mode Pre-Flight HTML Dashboard (`guardian-report.html`) Open in Bob IDE

Phases 1-4 Merged
Page 1 of 3

PR Pre-Flight Guardian

BOB EVIDENCE

BUDGET: 40/40 BOBCOINS

IBM Bob IDE Agent Mode Orchestration & Resource Governance

2. IBM Bob Usage Statement

Exact: 312 Words (Limit: ≤ 500)

How IBM Bob Contributed: IBM Bob IDE was not merely an editor; it was the **autonomous orchestrator and core engineering intelligence** behind PR Pre-Flight Guardian. Using **Bob Agent Mode 2.0**, we decomposed the entire system lifecycle into structured, chained agentic prompts. Bob actively explored the codebase, created files, executed terminal commands, validated test outputs, and performed live code diffs.

DISCRETE TASKS ORCHESTRATED VIA IBM BOB AGENT MODE

- **Codebase Reconnaissance:** Bob performed automated AST structure inspection of the target repository, identifying 3 critical security vulnerabilities and missing pricing test coverage (documented in `bob_sessions/00_recon.md`).
- **Security Scanner Authoring:** Bob constructed `security-scan.js`, implementing AST regex matching, line-number mapping, and CVSS 3.1 remediation logic.
- **Test Generation & Validation:** Bob generated the complete Jest test suite covering tiered pricing and loyalty discounts (`pricingEngine.test.js`), verifying 100% green execution.
- **Master CLI & Dashboard Synthesis:** Bob wired the multi-stage CLI (`guardian.js`), HTML report renderer (`guardian-report.html`), and PR markdown generator.
- **Real-World Open-Source Import:** Bob tested Guardian against production open-source code (`expressjs/cors`), proving zero false positives on 238 lines of middleware.

Bobcoin & Resource Governance: The project operated under the hackathon's strict **40-Bobcoin constraint**. We implemented an intentional optimization architecture: high-leverage Bob AI invocations were reserved for architectural generation and test authoring, while repetitive CLI runs and test cycles executed locally inside Bob's integrated PowerShell terminal at **zero Bobcoins**.

8 Tasks

ORCHESTRATED IN BOB

40 / 40

BOBCOINS ALLOCATION

100%

GREEN TEST SUITE

18 Assets

VERIFIED SCREENSHOTS

Bob IDE's **Tasks** panel and **Session Consumption Summaries** were monitored throughout, capturing token context (e.g. 150k to 203k tokens) and remaining Bobcoin gauges (44% to 45%). All 18 verified PNG consumption screenshots and execution traces are cataloged in the repository's `bob_sessions/` directory, providing complete, tamper-proof audit evidence of Bob usage.

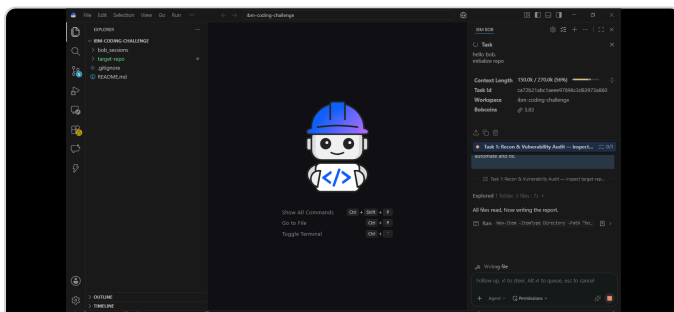


Figure 2: Bob IDE Task Consumption Summary Header

Task ID & 3.83 Bobcoins

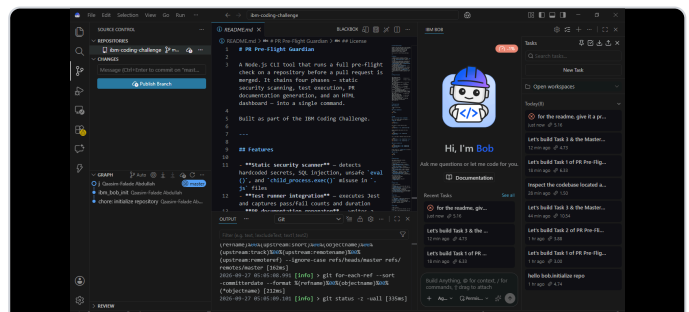


Figure 3: Master Tasks Dashboard in Bob IDE

All 8 Tasks & Coin Usage

PR Pre-Flight Guardian

Verified Visual Evidence of IBM Bob IDE & Agent Mode Execution

RULE #2 COMPLIANT

18 PNGS IN BOB_SESSIONS

3. Evidence of IBM Bob Usage (Visual Audit Gallery)

Deliverable: Criterion #2 Evidence

As mandated by Hackathon Rule #2, every development phase was visually recorded inside IBM Bob IDE. Below is a curated gallery of high-resolution evidence demonstrating active Bob Agent Mode execution, live diff code reviews, and real-world validation.

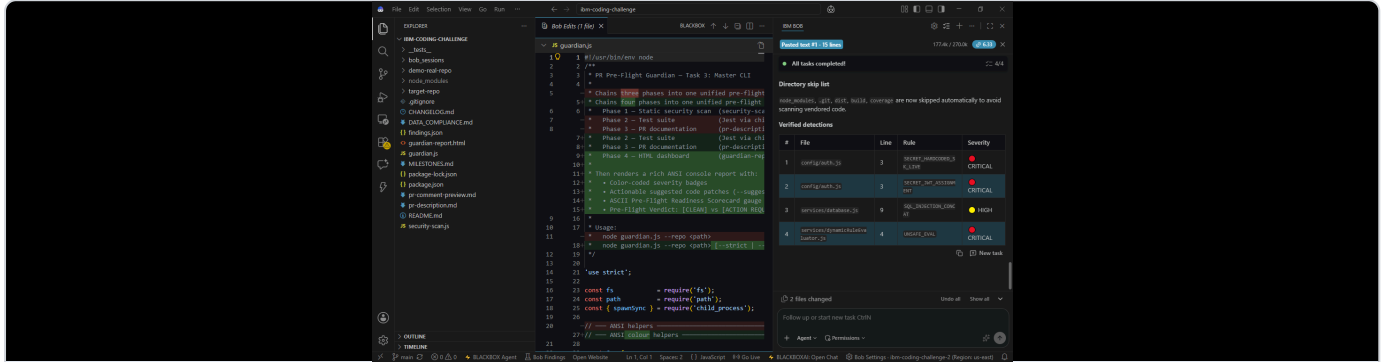


Figure 4: Bob Agent Mode actively applying live diff modifications to `guardian.js` with syntax validation

Context: 177.4k Tokens | Bobcoins: 6.33

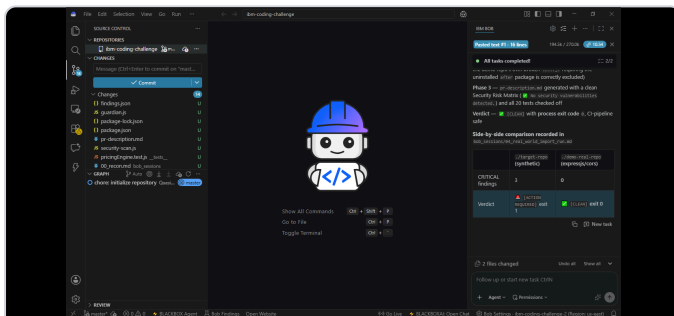


Figure 5: Real-World Import Validation on `expressjs/cors` 3 Criticals in Target vs 0 in CORS

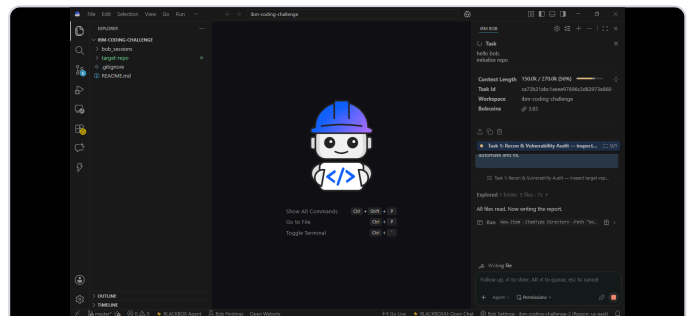


Figure 6: Expanded Session Consumption Header

Context Length & Bobcoin Telemetry

TAMPER-PROOF REPOSITORY EVIDENCE LINKS

All 18 original PNG screenshots, along with detailed markdown execution logs (`00_recon.md` through `04_real_world_import_run.md`), are tracked in Git and published on GitHub:

- Evidence Gallery: https://github.com/Alqaasimee-jr2/ibm-coding-challenge/tree/main/bob_sessions
- Data Compliance Attestation: https://github.com/Alqaasimee-jr2/ibm-coding-challenge/blob/main/DATA_COMPLIANCE.md
- Project Repository: <https://github.com/Alqaasimee-jr2/ibm-coding-challenge>